ExoDoom syscall interface specification

Derived from static analysis of doomgeneric source code

Document version 1.0 — Generated from github.com/ozkl/doomgeneric @ HEAD

1. The doomgeneric platform interface

doomgeneric requires exactly 6 functions to be implemented by the platform. These are the ONLY entry points between the Doom engine and the host environment. Everything else (rendering, game logic, AI, physics) is handled internally by the engine.

Function	Signature	What it does
DG_Init	`void DG_Init(void)`	Called once after DG_ScreenBuffer is allocated (640×400×4 bytes). Initialize framebuffer mapping and input.
DG_DrawFrame	`void DG_DrawFrame(void)`	Called by I_FinishUpdate after Doom writes a frame to DG_ScreenBuffer. Blit that buffer to the actual framebuffer.
DG_SleepMs	`void DG_SleepMs(uint32_t ms)`	Sleep for ms milliseconds. Busy-wait on PIT tick counter is fine.
DG_GetTicksMs	`uint32_t DG_GetTicksMs(void)`	Return monotonic milliseconds since boot. Used for game timing (35 tics/sec) and frame pacing.
DG_GetKey	`int DG_GetKey(int* pressed, unsigned char* key)`	Dequeue next key event. Returns 1 if event available (sets pressed=0/1 and key=doom keycode), 0 if queue empty.
DG_SetWindowTitle	`void DG_SetWindowTitle(const char* title)`	No-op on bare metal. Can print to serial for debug.

Important: DG_ScreenBuffer is a 640×400 pixel RGBA8888 buffer (pixel_t = uint32_t). Doom's internal rendering is 8-bit paletted; I_FinishUpdate in i_video.c converts palette indices to RGBA and writes to DG_ScreenBuffer before calling DG_DrawFrame. Your DG_DrawFrame only needs to copy/scale this buffer to the hardware framebuffer.

Mouse input is NOT part of the DG_* interface. The mouse code in i_input.c is commented out (it references SDL events). To add mouse support, patch I_GetEvent() to post ev_mouse events with data2=dx, data3=-dy by polling the mouse driver via syscall.

2. Complete libc dependency audit

The following is an exhaustive list of every standard C library function called by doomgeneric's 82 core source files, with call counts from static grep analysis. This determines what the freestanding libc shim must provide.

2.1 string.h — all required, no shortcuts

Function	Calls	Notes
`strlen`	64	Used everywhere. Trivial to implement.
`memset`	50	Used for clearing buffers, zero-init. Trivial.
`memcpy`	45	Used for framebuffer blitting, WAD data copying. Performance-sensitive — consider 32-bit aligned implementation.
`strdup`	16	Allocates + copies. Depends on malloc + strlen + memcpy.
`strcmp`	16	Standard string compare.
`strcasecmp`	9	Case-insensitive compare. NOT in C standard (POSIX). Used heavily in WAD/config parsing. Must implement.

`memcmp`	8	Byte comparison. Trivial.
`strncasecmp`	7	Case-insensitive compare with length limit. Also POSIX, not C standard.
`strchr`	7	Find character in string.
`strncmp`	6	Compare with length limit.
`strrchr`	5	Find last occurrence of character.
`strerror`	4	Returns string for errno value. Can return a static “unknown error” string.
`strstr`	3	Find substring. Used in config parsing.
`strncpy`	3	Copy with length limit.
`memmove`	3	Overlapping copy. Must handle src/dst overlap correctly.

2.2 stdlib.h — malloc is critical path

Key insight: Doom calls malloc exactly ONCE in I_ZoneBase() to allocate a 6 MiB zone, then manages memory internally via Z_Malloc. However, there are ~21 additional direct malloc calls scattered across WAD discovery, config parsing, string handling, and the screen buffer allocation. The libc malloc must work but does not need to be high-performance.

Function	Calls	Notes
`free`	153	Most calls are Z_Free (internal zone). ~20 are direct libc free().
`exit`	31	Called on fatal errors. Implement as halt loop.
`malloc`	21	One 6 MiB zone alloc + ~20 small allocs (strings, paths, structs).
`abs`	30	Integer absolute value. Trivial macro.
`atoi`	14	String to integer. Used for config/command-line parsing.
`atof`	2	String to float. Used only for mouse acceleration config. Can return 1.0 as stub.
`atexit`	4	Register cleanup functions. Implement as linked list (Doom already does this via I_AtExit).
`getenv`	3	Returns DOOMWADPATH/DOOMWADDIR. Return NULL — WAD is a multiboot module.
`system`	13	All behind #ifdef guards (Zenity error boxes). Stub as return -1.
`realloc`	3	Resize allocation. Implement as malloc+memcpy+free.
`calloc`	2	malloc + memset(0). Trivial wrapper.
`abort`	2	Abnormal termination. Implement as halt loop.
`qsort`	0 direct	Not called directly but may be pulled in. Implement a simple quicksort.

2.3 stdio.h — the hardest category

stdio.h is the most complex dependency. Doom uses printf extensively for debug output, and uses real FILE* I/O for three distinct purposes: (1) reading WAD files from disk, (2) reading/writing config files, and (3) reading/writing save games. Since the WAD is a multiboot module (already in memory), option A is to replace w_file_stdc.c with a memory-mapped reader. Config and save files need either a ramdisk-backed or ATA-backed FILE* implementation.

Function	Calls	Notes
`printf`	143	Redirect to serial output via exo_serial_write syscall. Most output is init/debug messages.

fprintf	92	Mostly fprintf(stderr, ...). Redirect to serial. A few write to FILE* for config save.
fopen	21	Opens WAD, config, save files. Must return a FILE* struct backed by ramdisk or ATA.
fclose	21	Close file handle. Free FILE* struct.
fwrite	16	Write config and save game data.
fread	14	Read WAD data and save games.
remove	13	Delete files (old save games). Implement for ramdisk; no-op is acceptable initially.
sscanf	8	Parse formatted strings. Used in config parsing (%d, %f, %x formats). Must implement.
puts	6	Print string + newline to stdout. Route to serial.
ftell	6	Return current file position.
fflush	6	Flush buffered output. No-op for unbuffered serial. Needed for FILE* writes.
fseek	5	Seek to position in file. Used by WAD reader (SEEK_SET, SEEK_END).
vsnprintf	5	Format string to buffer with length limit. Doom has its own M_vsnprintf wrapper that calls this.
putchar	3	Write single char. Route to serial.
rename	3	Rename save file. Implement for ramdisk or stub.
feof	2	Check end-of-file. Return based on FILE* position.
vfprintf	1	Used in I_Error. Format + print to stderr (serial).
sprintf	1	Unsafe format to buffer. Wrapper around vsnprintf.
snprintf	1	Format to buffer with length. Core function; vsnprintf does the work.
fgets	1	Read line from file. Used in config parsing.
fileno	1	Behind #ifdef ORIGCODE. Stub as return -1.

2.4 math.h — almost entirely avoidable

Doom uses fixed-point arithmetic internally. The 187 calls to floor() are actually Doom's FIXED_TO_INT macro which is integer division, not floating-point. The ONLY real floating-point math is in R_InitTables() in r_main.c, which runs ONCE at startup to build sine/tangent lookup tables from 3 trig calls. Options: (A) implement sin/cos/tan/atan with a small Taylor series or lookup table, or (B) precompute the tables and hardcode them.

Function	Calls	Notes
floor	187	NOT floating-point. Doom uses this for fixed-point→int conversion. Implement as: (int)(x) for positive, (int)(x)-1 for negative with fractional part.
atan	1	Called once in R_InitTables. Builds FINEANGLES lookup table.
tan	1	Called once in R_InitTables. Builds finetangent table.
sin	1	Called once in R_InitTables. Builds finesine table.
fabs	1	Floating-point absolute value. Used once in mouse acceleration check. Trivial.

2.5 ctype.h, stdarg.h, assert.h, errno.h, limits.h

ctype.h: isspace (12), toupper (11), tolower (7), isprint (1), isalpha (1). All trivial ASCII range checks.

stdarg.h: va_start (5), va_end (5), va_arg (2). Provided by the compiler (__builtin_va_*). No implementation needed.

assert.h: assert (6). Implement as macro that prints file:line to serial and halts on failure.

errno.h: errno (8) used as a global int. Only EISDIR is tested (in M_FileExists). Define errno as a global int and EISDIR as a constant.

limits.h: INT_MAX used in I_GetPaletteIndex. Define as 0x7FFFFFFF.

inttypes.h: PRIx64 macro. Define as "llx" or stub.

fcntl.h: Included but only O_RDONLY etc. are used in platform-specific ports. Can be an empty header.

2.6 POSIX / platform functions to stub

Function	Used in	Strategy
`getenv`	d_iwad.c, m_misc.c	Return NULL. WAD is loaded via multiboot module, not filesystem path.
`mkdir`	m_misc.c	No-op or create ramdisk directory entry. Only called for save game directory.
`system`	i_system.c	Return -1. Only used for Zenity error dialogs on Linux.
`isatty / fileno`	i_system.c	Behind ORIGCODE guard. Stub: return 0.
`strcasecmp`	9 files	POSIX, not C standard. Implement as case-insensitive strcmp loop.
`strncasecmp`	2 files	POSIX, not C standard. Implement as case-insensitive strncmp.

3. Exokernel syscall specification

The following syscall interface is derived bottom-up from the actual requirements of doomgeneric, the libc shim, and the LibOS infrastructure. Each syscall is mapped to the Doom feature that requires it.

3.1 Syscall calling convention

Entry: int 0x80. Syscall number in EAX. Arguments in EBX, ECX, EDX, ESI, EDI (up to 5 arguments). Return value in EAX. Negative return = error code. The kernel saves/restores all other registers.

```
// LibOS-side syscall stub example:
static inline int32_t exo_syscall1(uint32_t num, uint32_t arg1) {
    int32_t ret;
    __asm__ volatile("int $0x80" : "=a"(ret) : "a"(num), "b"(arg1));
    return ret;
}
```

3.2 Syscall table

#	Syscall	Category	Description + Doom usage
0	**`exo_page_alloc()`**	Memory	Allocate one 4K physical page. Returns physical address, or -ENOMEM. Used by LibOS heap allocator backing malloc/Z_Malloc.
1	**`exo_page_free(paddr)`**	Memory	Free a physical page. Returns 0 or -EINVAL. Used by LibOS heap to return pages.
2	**`exo_page_map(vaddr, paddr, flags)`**	Memory	Map physical page at virtual address in caller's page directory. flags: read/write/user. Returns 0 or -EINVAL/-EFAULT. Used to build LibOS heap and map framebuffer.

3	`exo_page_unmap(vaddr)`	Memory	Unmap a virtual page. Returns 0 or -EINVAL.
4	`exo_fb_acquire(info_out)`	Framebuffer	Write framebuffer info (phys_addr, width, height, pitch, bpp) to info_out struct. LibOS then calls exo_page_map to map it. Used by DG_Init. Returns 0 or -EBUSY if another LibOS holds the FB.
5	`exo_get_ticks()`	Timer	Return uint32_t milliseconds since boot. Zero arguments. Used by DG_GetTicksMs and DG_SleepMs (busy-wait loop). Called ~35 times/sec by the game loop.
6	`exo_kbd_poll(event_out)`	Input	Dequeue next keyboard event into event_out struct {uint8_t pressed; uint8_t scancode}. Returns 1 if event available, 0 if empty. Used by DG_GetKey.
7	`exo_mouse_poll(state_out)`	Input	Write accumulated mouse state {int16_t dx; int16_t dy; uint8_t buttons} to state_out, then reset accumulators. Returns 0. Used by patched I_GetEvent to post ev_mouse.
8	`exo_serial_write(buf, len)`	Debug	Write len bytes from buf to COM1. Returns bytes written. Used by printf/fprintf shim. Validates buf is in user address space.
9	`exo_file_open(path, mode)`	File I/O	Open a file on the ramdisk/ATA filesystem. mode: 0=read, 1=write, 2=read+write. Returns file descriptor (>= 0) or negative error. Used by fopen shim.
10	`exo_file_close(fd)`	File I/O	Close file descriptor. Returns 0 or -EBADF. Used by fclose shim.
11	`exo_file_read(fd, buf, count)`	File I/O	Read up to count bytes from fd into buf. Returns bytes read, 0 at EOF, or negative error. Used by fread shim.
12	`exo_file_write(fd, buf, count)`	File I/O	Write count bytes from buf to fd. Returns bytes written or negative error. Used by fwrite shim.
13	`exo_file_seek(fd, offset, whence)`	File I/O	Seek to position. whence: 0=SEEK_SET, 1=SEEK_CUR, 2=SEEK_END. Returns new position or negative error. Used by fseek/ftell shim.
14	`exo_file_stat(path, size_out)`	File I/O	Write file size to *size_out. Returns 0 or -ENOENT. Used by M_FileExists (fopen check) and M_FileLength.
15	`exo_file_remove(path)`	File I/O	Delete a file. Returns 0 or -ENOENT. Used by remove() for old save games.
16	`exo_file_rename(old, new)`	File I/O	Rename a file. Returns 0 or negative error. Used by rename() for save game rotation.
17	`exo_sound_tone(freq, dur_ms)`	Sound	Play a tone on the PC speaker at freq Hz for dur_ms milliseconds. Non-blocking (kernel manages PIT ch2). Returns 0. Used by I_StartSound shim.

18	`exo_sound_stop()`	Sound	Silence the PC speaker immediately. Returns 0. Used by I_StopSound shim.
19	`exo_yield()`	Scheduling	Cooperatively yield CPU to next runnable LibOS context. Returns when rescheduled. Used by shell LibOS and optionally by DG_SleepMs.
20	`exo_exit(code)`	Lifecycle	Terminate calling LibOS. Frees all pages, closes all files, removes from scheduler. Does not return.

21 syscalls total. This is the complete interface needed to run Doom with save/load, config, sound, and cooperative multitasking.

4. Architectural decision: file I/O strategy

Doom uses FILE* I/O for three distinct purposes. Each can be handled differently:

4.1 WAD file reading (w_file_stdc.c)

The WAD is loaded as a GRUB multiboot module and is already in physical memory. Option A (recommended): replace w_file_stdc.c with a memory-mapped reader that returns pointers into the module's mapped memory. fopen("freedoom2.wad") returns a fake FILE* backed by the in-memory WAD data. fseek/fread operate on offsets into the mapped region. This avoids any actual file I/O for the largest data source. Option B: funnel through exo_file_* syscalls with the kernel maintaining a "virtual file" backed by the multiboot module memory.

4.2 Config files (m_config.c)

Doom reads/writes default.cfg for key bindings, video settings, etc. This is a small text file (< 4 KB). A ramdisk is sufficient initially (config lost on reboot). For persistence, the ATA driver can store it in a reserved disk sector. Goes through exo_file_* syscalls.

4.3 Save games (g_game.c)

Save files are ~200 KB each. Doom supports 6 save slots (doomsav0.dsg through doomsav5.dsg) plus temp/recovery files. This needs real read/write/seek semantics. Ramdisk for Sprint 10, ATA-backed for Sprint 11. Goes through exo_file_* syscalls. The save path also uses remove() and rename() for safe file rotation.

Recommended approach: implement a minimal ramdisk filesystem in the kernel (flat list of named files with read/write/seek), exposed through exo_file_* syscalls. The LibOS libc shim translates FILE to file descriptors. WAD reading uses the memory-mapped shortcut. Later, add ATA persistence behind the same syscall interface.*

5. Memory allocation pattern

Doom's memory usage is well-defined and predictable:

Zone allocator: I_ZoneBase() calls malloc(6 * 1024 * 1024) once to get 6 MiB. All game-internal allocations (level data, sprites, sounds, intermission screens) go through Z_Malloc which manages this zone internally. If 6 MiB fails, Doom retries with decreasing sizes down to the MIN_RAM of 6 MiB (then crashes). On a 256 MiB QEMU VM this will never fail.

Screen buffer: doomgeneric_Create() calls malloc(640 * 400 * 4) = 1,024,000 bytes for DG_ScreenBuffer.

Scattered small allocs: ~20 direct malloc calls for strings, paths, config values, temp buffers. Largest is ~4 KB. Total < 100 KB.

Total LibOS heap requirement: ~8 MiB is safe. This means the LibOS page allocator needs about 2,048 pages (8 MiB / 4 KB) from the exokernel's physical page pool.

6. Sound architecture

When FEATURE_SOUND is not defined (the default for non-SDL builds), the sound_module pointer is NULL and all I_StartSound/I_StopSound calls are no-ops. This means Doom will run silently with zero sound code. To add PC speaker sound, there are two options:

Option A (minimal): Keep FEATURE_SOUND undefined. Doom runs silently. No sound syscalls needed.

Option B (PC speaker): Implement a sound_module_t with Init/StartSound/StopSound/Update that maps Doom SFX lump data to PC speaker tone frequencies and calls exo_sound_tone/exo_sound_stop. Define FEATURE_SOUND and register the module. The mapping from Doom's 8-bit PCM sound lumps to single-frequency tones is lossy but recognizable.

This specification was generated by analyzing the doomgeneric repository at github.com/ozkl/doomgeneric via automated grep-based static analysis of 82 core C source files. Function call counts are approximate (grep matches include declarations, comments, and string literals in some cases) but representative of actual usage patterns.